AES vs 3DES vs RSA

Marcin Kocot

Zadanie wykonano w języku Kotlin, przy użyciu build-toola Bazel.

Implementacja

Wykorzystano bibliotekę `javax.crypto.Cipher` - pozwala ona na łatwe użycie wszystkich najpopularniejszych algorytmów szyfrujących. W porównaniu udział brały:

- AES (`AES/CBC/PKCS5Padding`)
  - AES128
  - AES192
  - AES256
- 3DES (`TripleDES/CBC/PKCS5Padding`)
- RSA
  - RSA512
  - RSA1024
  - RSA2048
  - RSA4096

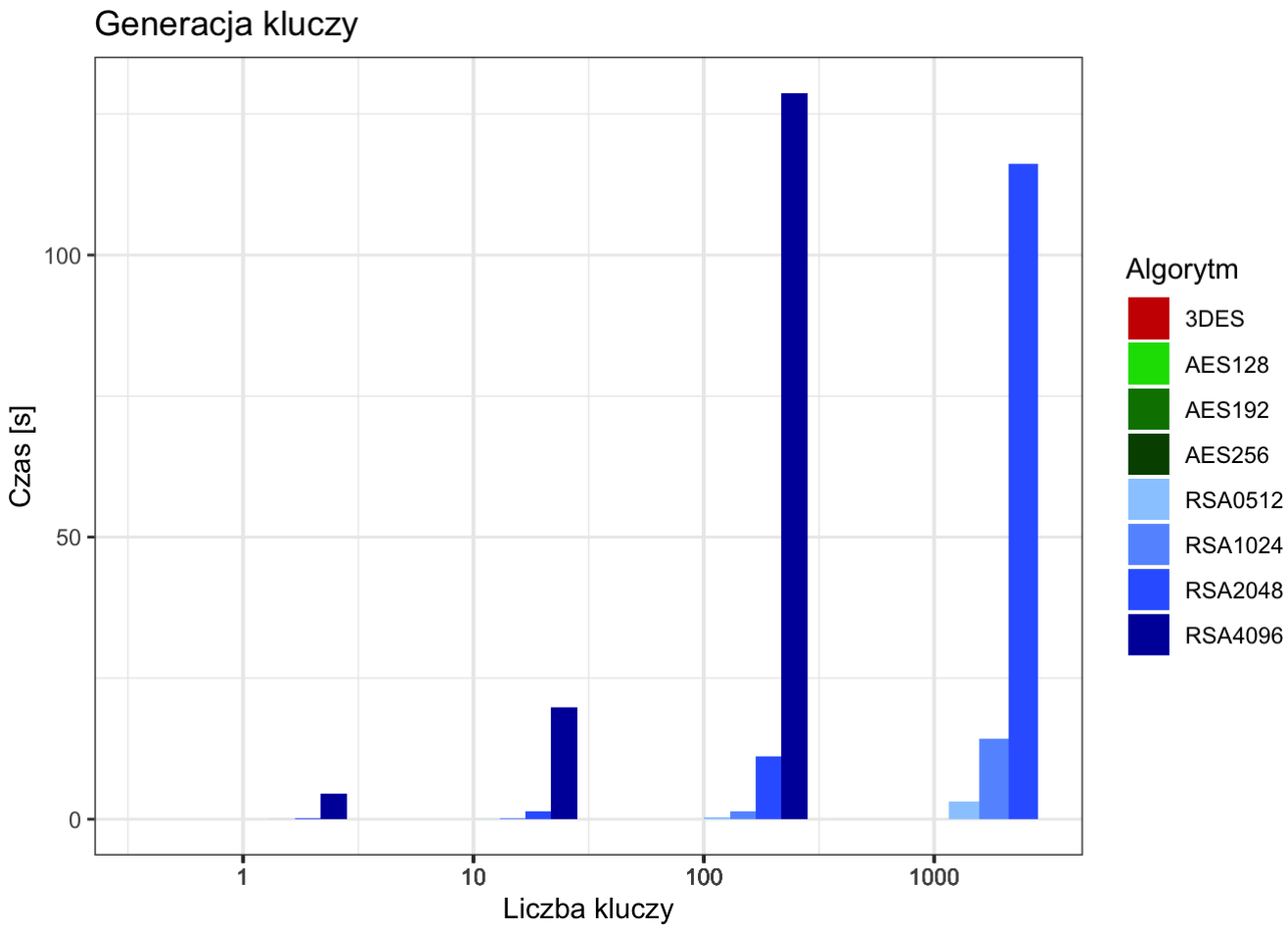
Utworzono abstrakcyjną klasę odpowiedzialną za pomiary, a implementacje poszczególnych algorytmów dostarczały tylko metod generujących klucze, syfrujących i deszyfrujących. Dzięki temu każdy lagorytm był mierzony w dokładnie ten sam sposób. Wszystkie pomiary były przeprowadzone na procesorze Apple M1 z 16GB pamięci RAM. Przy wszystkich uruchomieniach maszyna wirtualna Javy (JVM) miała tę samą konfigurację i stałe zużycie zasobów - dlatego też pod uwagę będzie brany głównie czas.

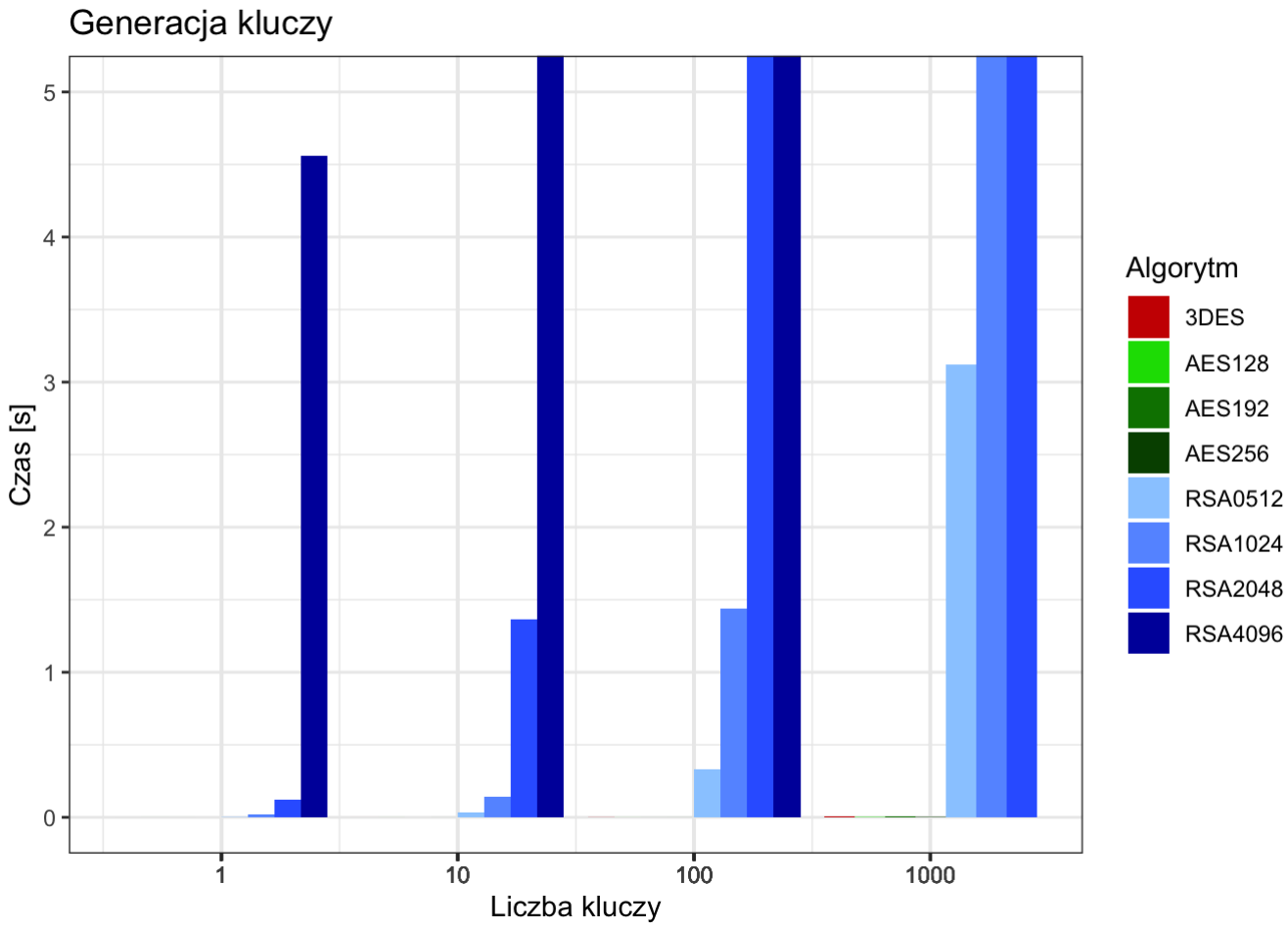
Pomiary

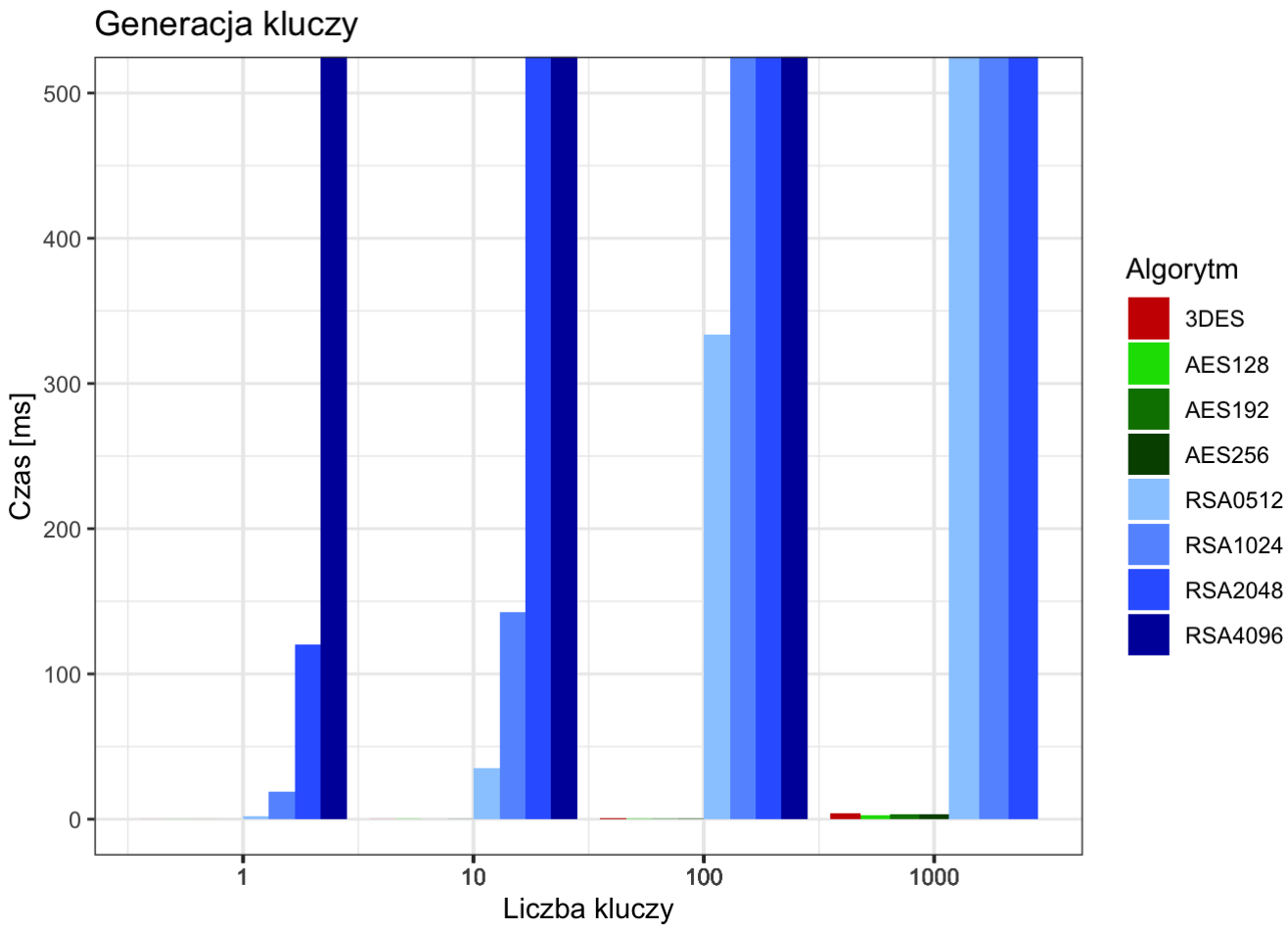
Żeby zminimalizować wpływ czynników losowych na pomiary, każdy przeprowadzono dziesięć razy, a wyniki uśredniono.

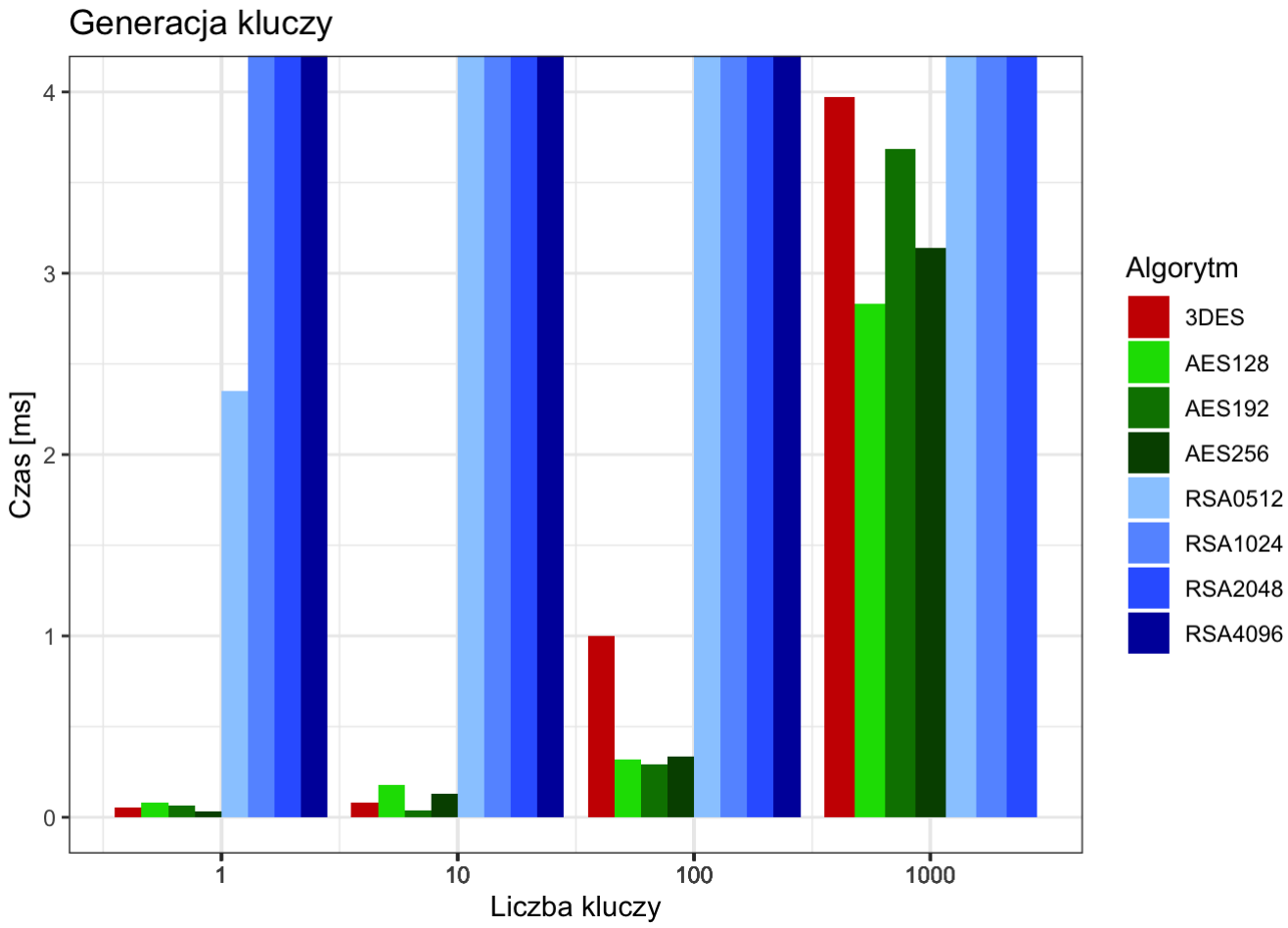
Generacja kluczy

Pomiary przeprowadzono dla generacji 1, 10, 100 i 1000 kluczy. Wyjątkiem jest `RSA4096` , dla którego pominięto największy test (ponieważ trwałby on bardzo długo).


Generacja kluczy
Czas [s]
100
50
0
1
10
100
1000
Liczba kluczy
Algorytm
3DES
AES128
AES192
AES256
RSA0512
RSA1024
RSA2048
RSA4096


Generacja kluczy
Czas [s]
5
4
3
2
1
0
1
10
100
1000
Liczba kluczy
Algorytm
3DES
AES128
AES192
AES256
RSA0512
RSA1024
RSA2048
RSA4096



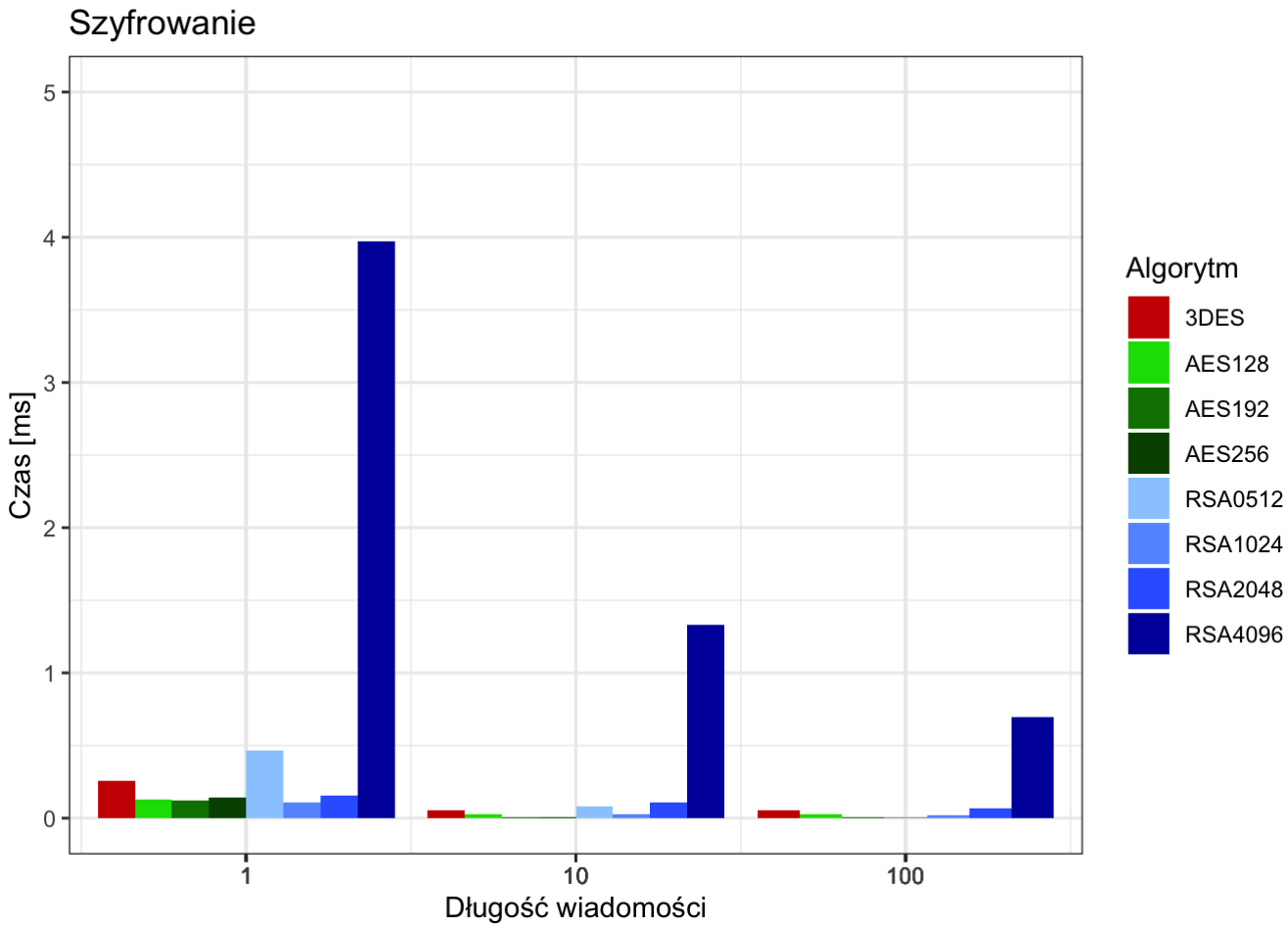


Biorąc pod uwagę czas potrzebny na generację kluczy, szyfry symetryczne (`3DES` i `AES`) wypadają dużo lepiej. Wynika to z faktu, że dla nich generacja klucza wymaga wyłącznie utworzenia odpowiednio długiego ciągu bajtów, co jest w stanie wykonać dowolny generator liczb losowych. `RSA` musi znaleźć odpowiednio

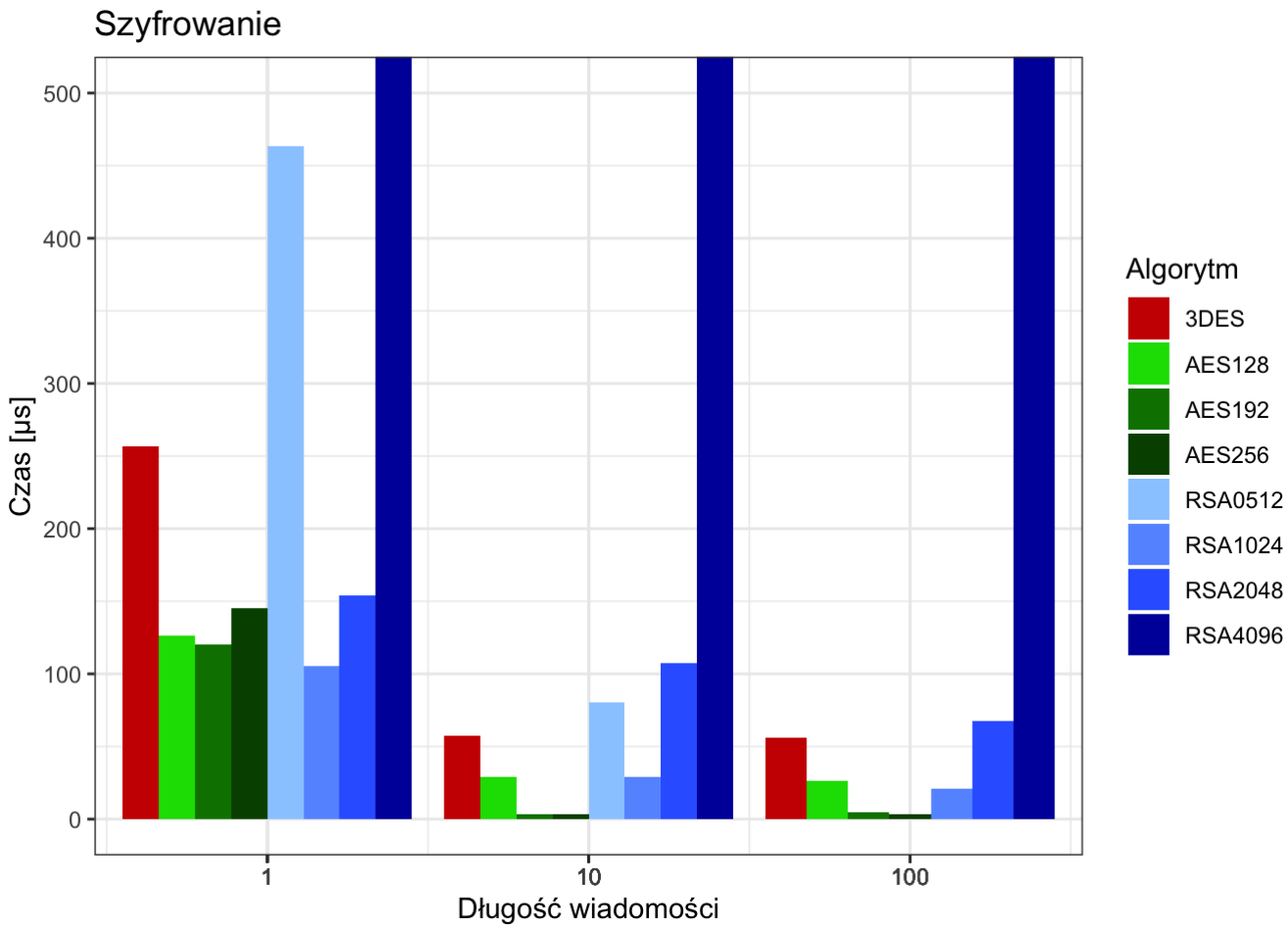
duże liczby pierwsze, co zajmuje dużo więcej czasu. Ciekawą obserwacją jest fakt, że długość generacji klucza `AES` zdaje się nie zależeć od jego długości.

Szyfrowanie

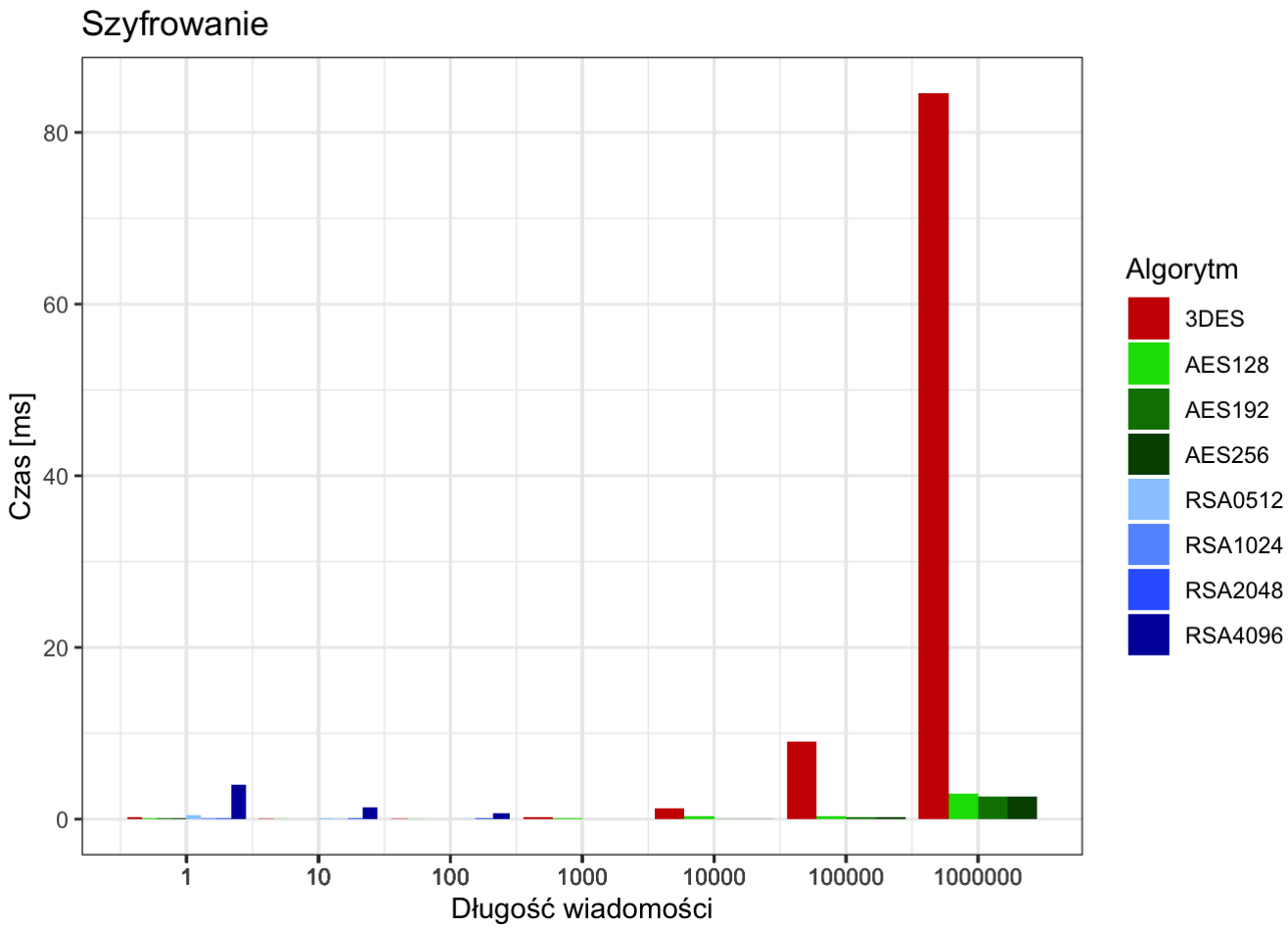
Pomiary przeprowadzono dla 1, 10, (…), 100 tys. i miliona znaków długości tekstu do zaszyfrowania. Niestety, bez ręcznego podziały wiadomości na bloki, testowanie `RSA` na wiadomościach o długości tysiąca znaków lub więcej nie było możliwe.

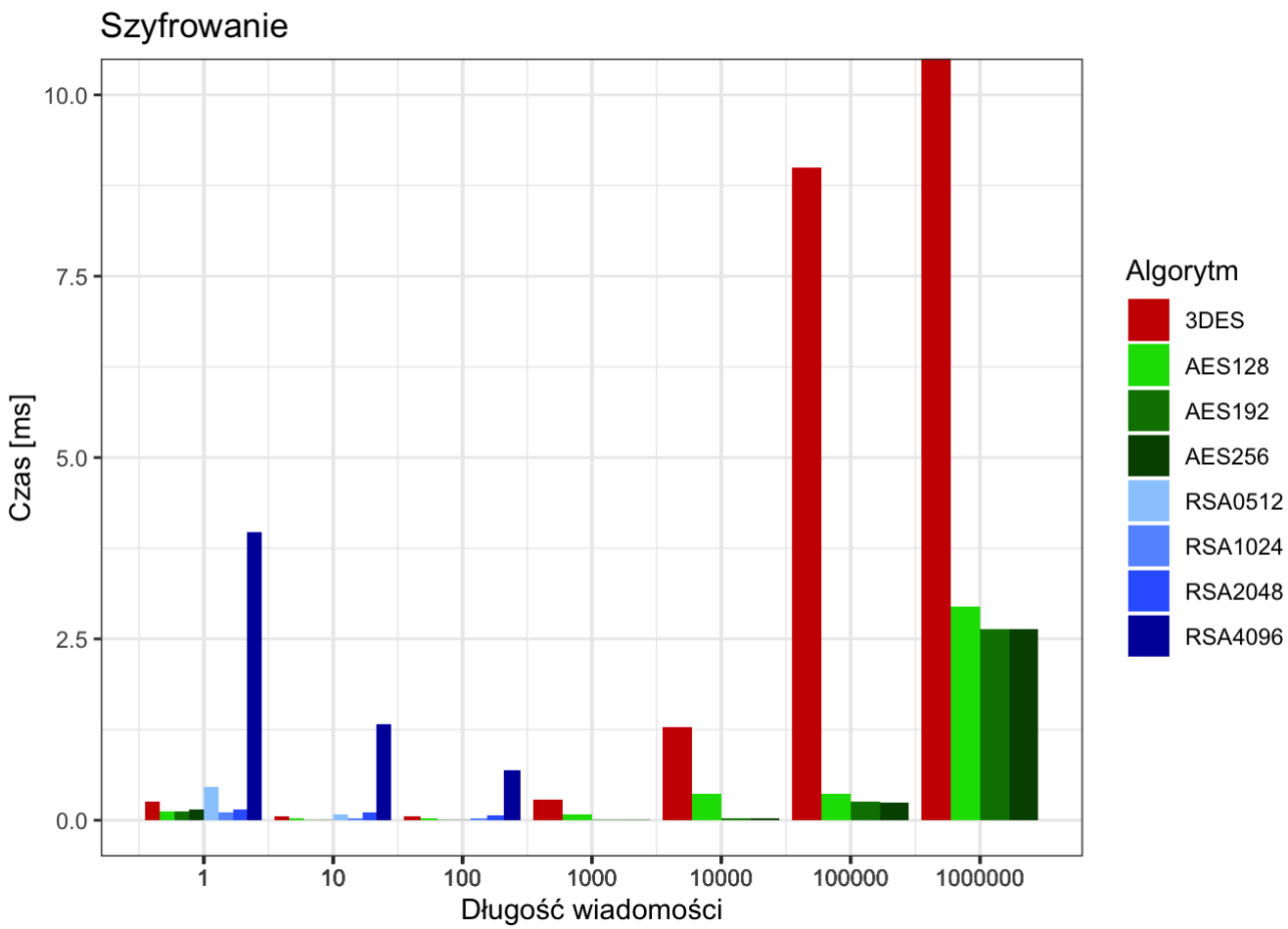


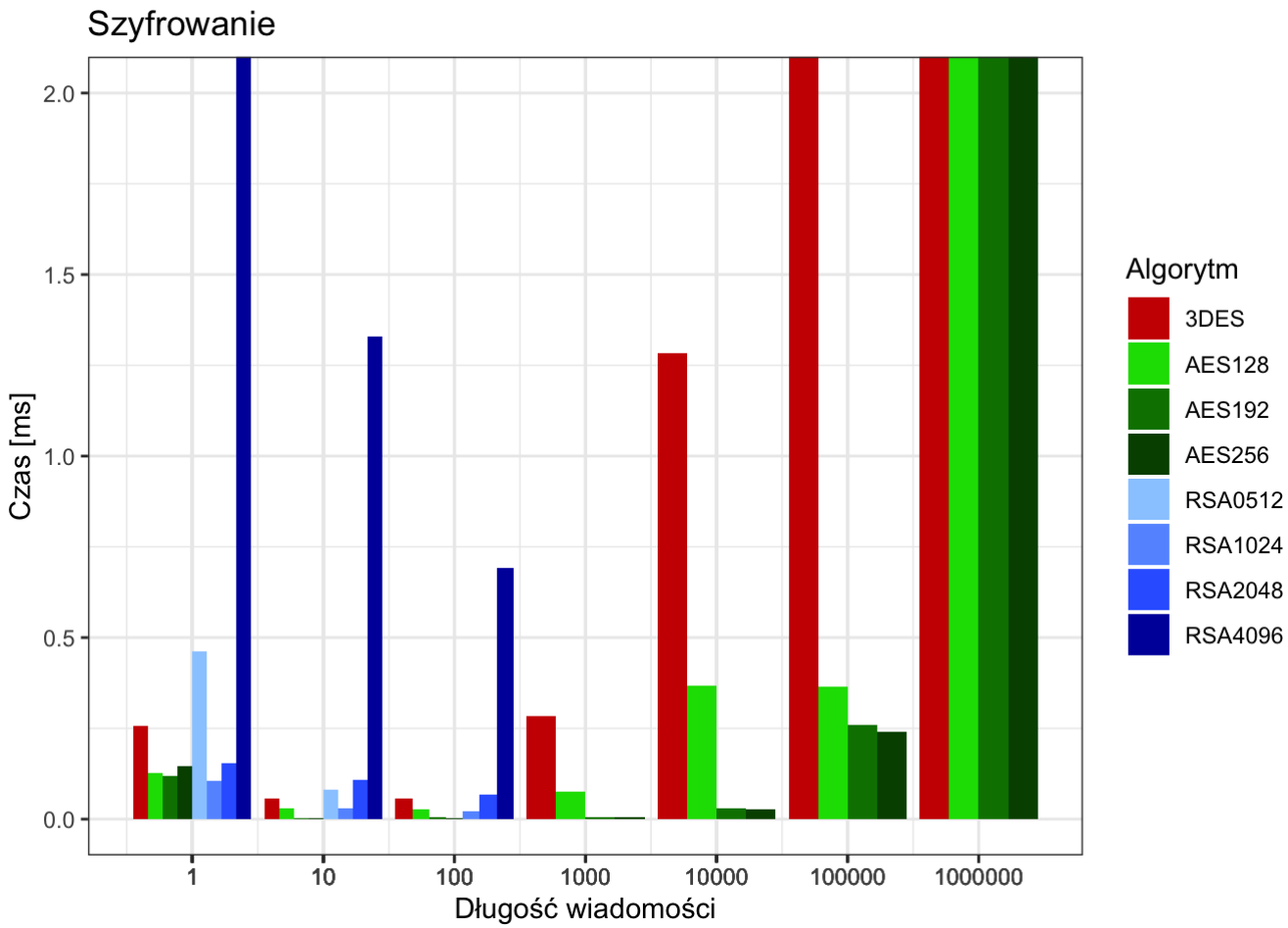
Możliwym powodem dlaczego szyfrowanie `RSA` jest szybsze dla dłuższych wiadomości jest fakt, że wiadomości wymagają paddingu - im krótsza wiadomość, tym potrzeba go więcej.



Szyfrowanie dla `RSA` (oprócz `RSA4096`) trwało porównywalnie do szyfrów symetrycznych. Nie powinno się jednak wysnuwać wniosku, że są one tak samo szybkie. Załóżmy, że skoro zaszyfrowanie długich wiadomości za pomocą `RSA` wymaga podziału na bloki i przetworzenia ich osobno, zaszyfrowanie dziesięciokrotnie dłuższej wiadomości zajmie dziesięciokrotność czasu.


Szyfrowanie
Czas [ms]
80
60
40
20
0
1
10
100
1000
10000
100000
1000000
Długość wiadomości
Algorytm
3DES
AES128
AES192
AES256
RSA0512
RSA1024
RSA2048
RSA4096


Szyfrowanie
Czas [ms]
10.0
7.5
5.0
2.5
0.0
1
10
100
1000
10000
100000
1000000
Długość wiadomości
Algorytm
3DES
AES128
AES192
AES256
RSA0512
RSA1024
RSA2048
RSA4096



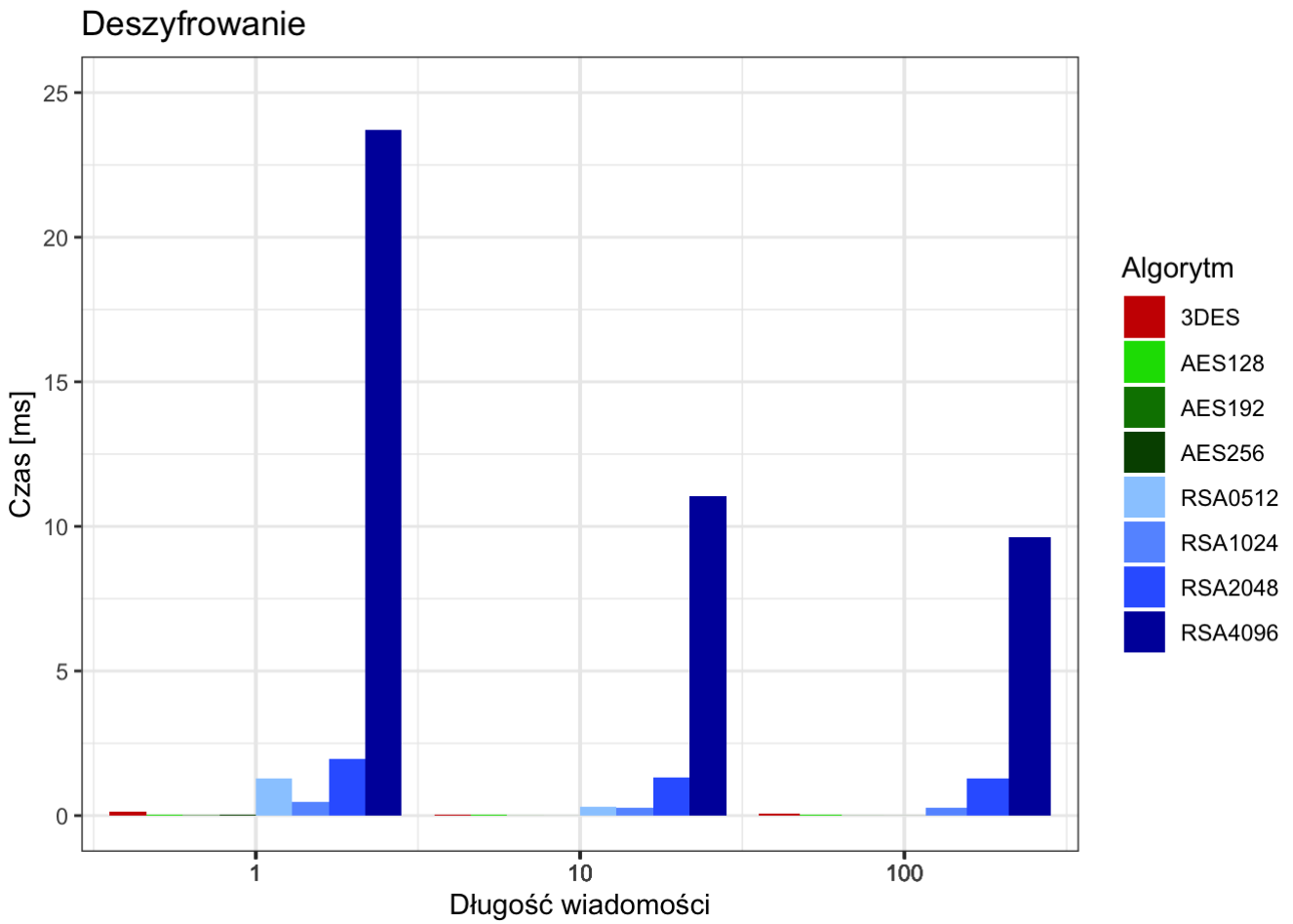
Idąc tym rozumowaniem, zaszyfrownie wiadomości o długości miliona znaków zajęłoby:

- 84,6 milisekund używając `3DES`
- 2,6 milisekundy za pomocą `AES256`
- 212 milisekund algorytmem `RSA1024`
- 670 milisekund przy użyciu `RSA2048`
- 6,9 sekundy (6926 milisekund) wykorzystując `RSA4096`

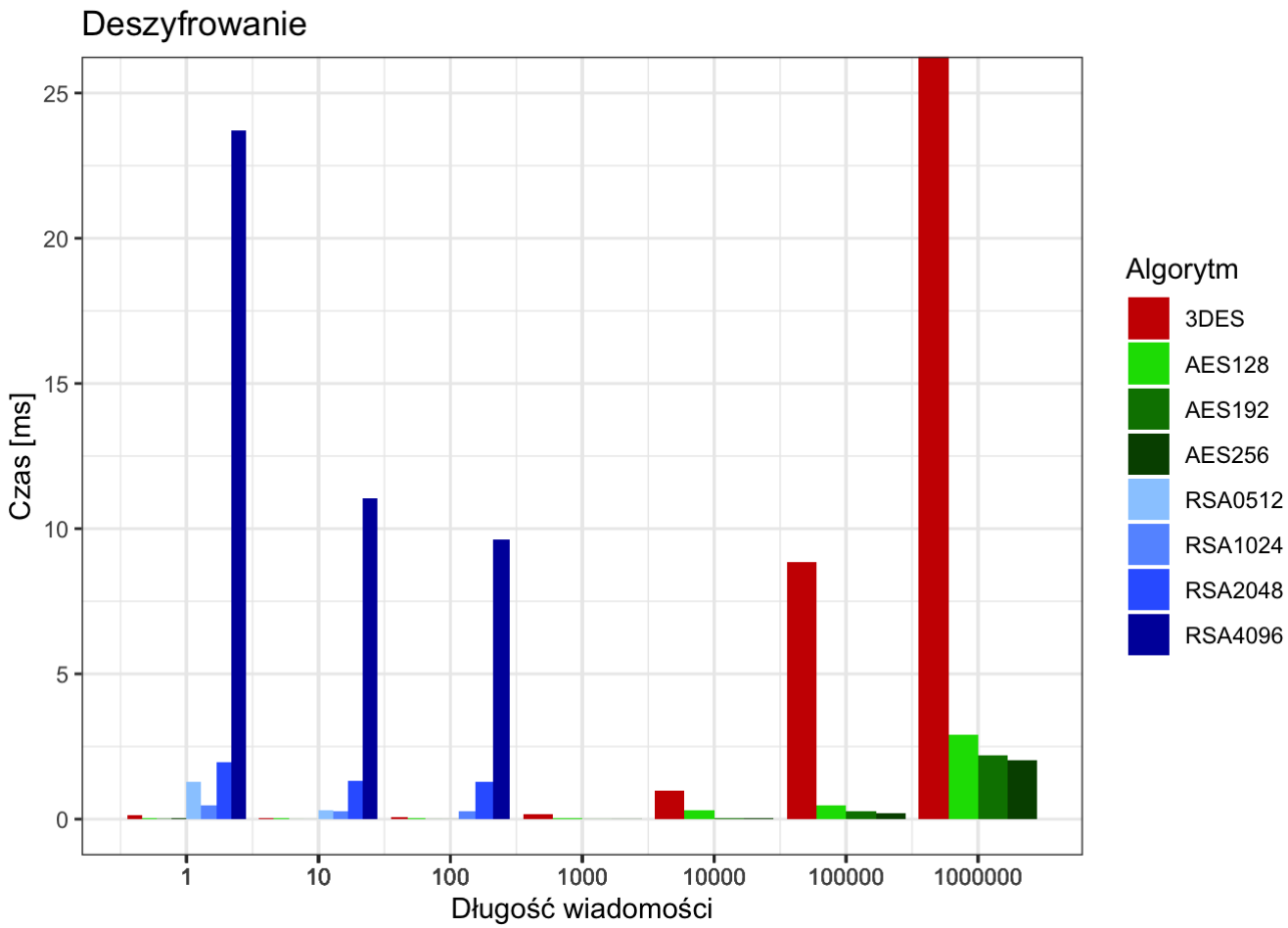
Dlatego też pod względem szybkości szyfrowania szyfry symetryczne wypadają dużo lepiej.

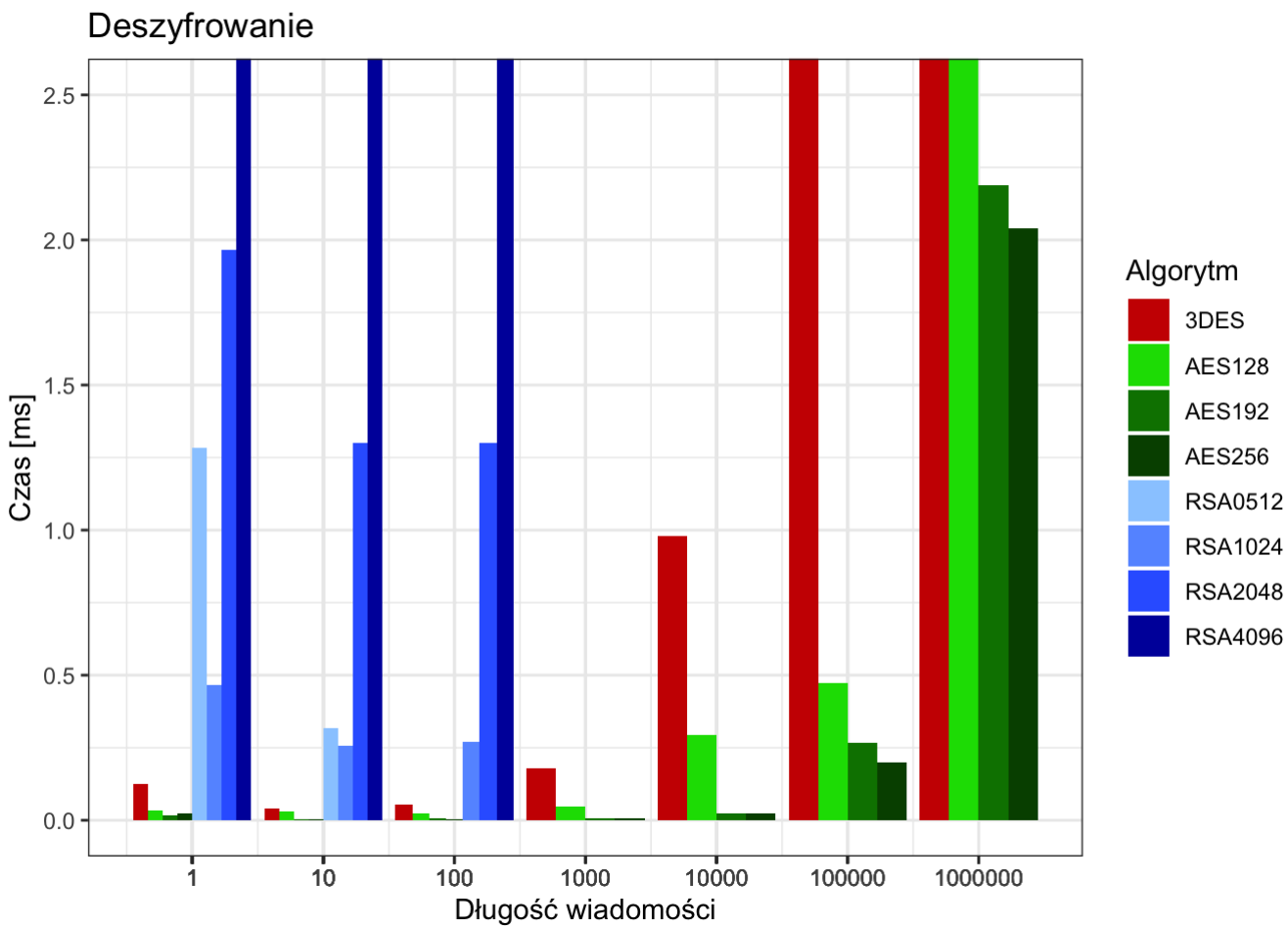
Deszyfrowanie

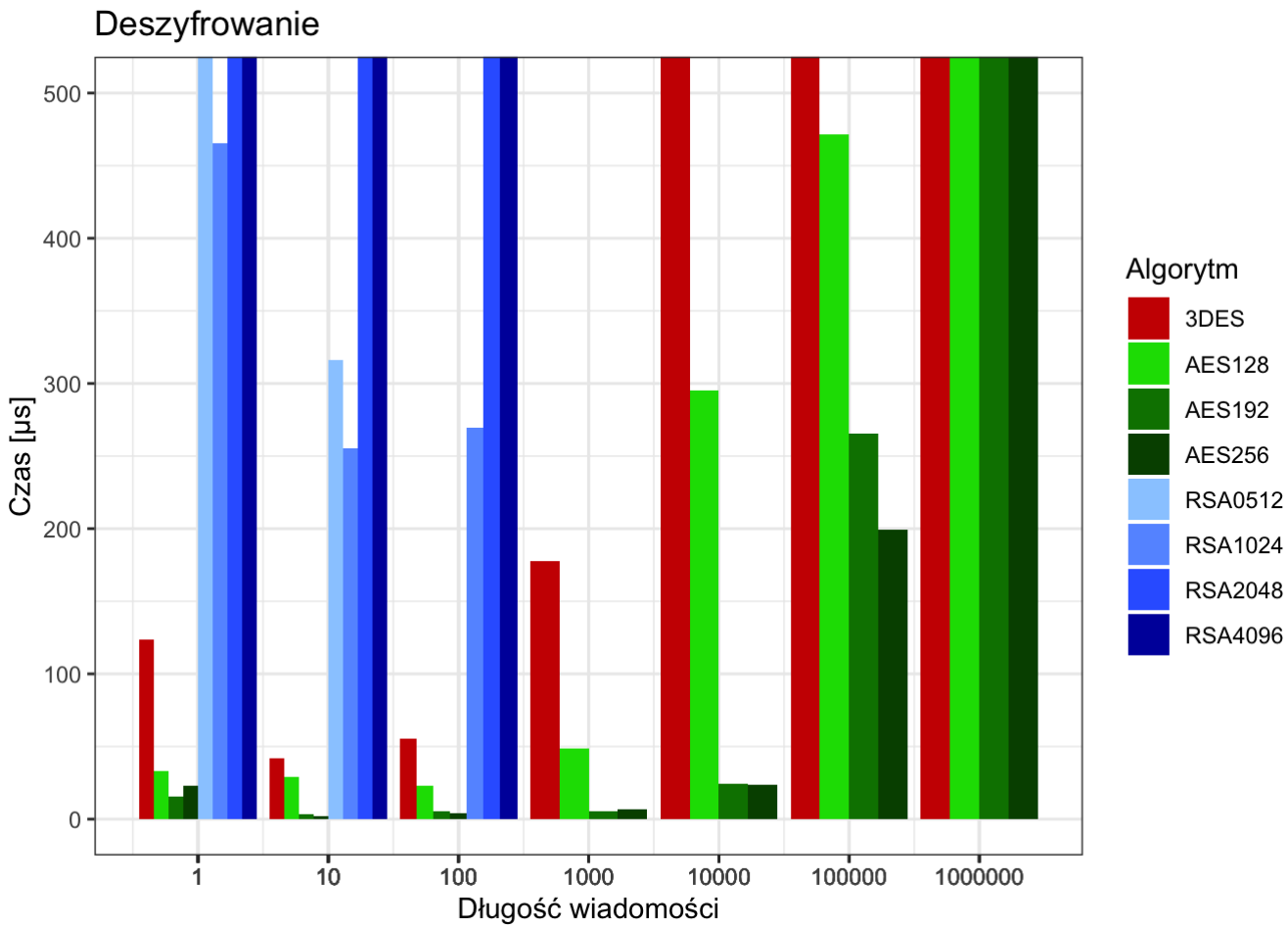
Przetestowano te same długości wiadomości co w przypadku szyfrowania.

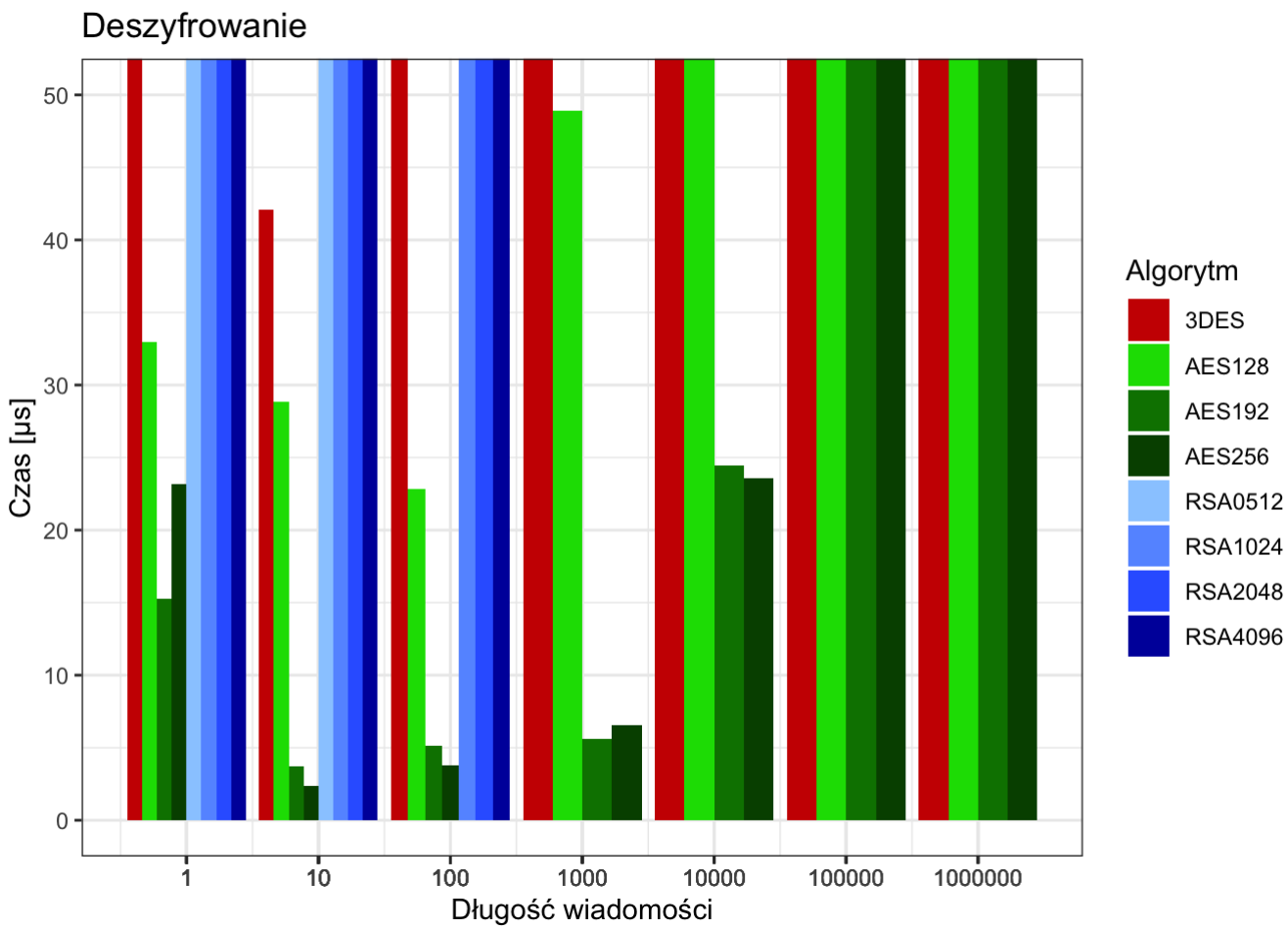


Zachodzi tu ta sama zależność co przy szyfrowaniu - krótsze wiadomości wymagają więcej czasu. Dodatkowo, odszyfrowywanie za pomocą algorytmu `RSA` jest dużo (ponad pięciokrotnie) dłuższe od szyfrowania - wynika to z faktu, że ta operacja jest w tym algorytmie bardziej złożona, a klucz prywatny jest dłuższy od publicznego. Przeprowadźmy tu tę samą analizę co przy szyfrowaniu:


Deszyfrowanie
Czas [ms]
25
20
15
10
5
0
1
10
100
1000
10000
100000
1000000
Długość wiadomości
Algorytm
3DES
AES128
AES192
AES256
RSA0512
RSA1024
RSA2048
RSA4096


Deszyfrowanie
Czas [ms]
2.5
2.0
1.5
1.0
0.5
0.0
1
10
100
1000
10000
100000
1000000
Długość wiadomości
Algorytm
3DES
AES128
AES192
AES256
RSA0512
RSA1024
RSA2048
RSA4096





Tutaj sytuacja jest inna niż w przypadku szyfrowania - asymetryczny wymagał znacznie więcej czasu niż symetryczne. Przeliczając czasy w ten sam sposób co przy szyfrowaniu uzyskano następujące wyniki dla wiadomości o długości miliona znaków:

- 84,5 milisekund używając `3DES`
- 2 milisekundy za pomocą `AES256`
- 2,7 sekundy algorytmem `RSA1024`
- 13 sekund przy użyciu `RSA2048`
- 1 minutę 36 sekund (96110 milisekund) wykorzystując `RSA4096`

Wnioski

Zarówno w przypadku generacji kluczy, szyfrowania jak i deszyfrowania, algorytm `RSA` był zawsze wolniejszy od algorytmów `AES` i `3DES`, czasem nawet o całe rzędy wielkości - jest to zgodne z analizami w artykułach naukowych przedstawionych na zajęciach. Przyjmując czas jako jedyne kryterium, ukazuje to szyfry symetryczne jako dużo lepsze. Jeśli jednak weźmie się pod uwagę także różnicę w zastosowaniach tych algorytmów, uznanie któregokolwiek za lepszy jest trudne. Na podstawie powyższych wyników można jednak uargumentować zasadność używania szyfrów hybrydowych - symetrycznych, z kluczem wysyłanym za pomocą asymetrycznego. Łączy to, mówiąc kolokwialnie, “najlepsze z obu światów”, jako że można go zastosować tak jak szyfr asymetryczny, ale korzysta z zaobserwowanej tutaj wydajności algorytmów symetrycznych.

Jeśli przyjrzymy się jak w tym porównaniu wychodzą szyfry symetryczne, to w większości pomiarów wszystkie rodzaje algorytmu `AES` były szybsze od `3DES`, z różnicą dochodzącą do dwudziestokrotnej. Jako że należą do tej samej rodziny, szyfr `AES256` można uznać za najlepszy z przetestowanych - działa szybko, a jest znacznie bezpieczniejszy niż jego mniejsze odpowiedniki (`AES128` i `AES196`).